J. File System (OSNo10)

📖 รู้จักส่วนนี้ก่อนอ่าน

ความหมาย: File System = ผู้จัดการไฟล์บนดิสก์ — ตั้งชื่อ เก็บ ดึง แก้ไข จัด metadata และกำหนดสิทธิ์การเข้าถึง โดยที่เราไม่ต้องรู้เลยว่าข้อมูลอยู่เซกเตอร์ไหนของฮาร์ดดิสก์

ความสำคัญกับข้อสอบ: เทียบ FAT32 (ระบบไฟล์เก่า ไฟล์เกิน 4GB ไม่ได้)/NTFS (ระบบไฟล์ของ Windows)/ext4/exFAT (FAT32 ตัวใหม่ ไฟล์ใหญ่ได้)/APFS (ระบบไฟล์ของ Apple), วิธีจัดสรรบล็อก 3 แบบ, คำนวณ inode (โครงสร้างเก็บข้อมูลไฟล์ของยูนิกซ์), hard link vs symbolic link, สิทธิ์ rwx → ส่วนใหญ่ออกถูก/ผิด + โจทย์คำนวณสั้น ๆ

ใช้จริงในคอมพิวเตอร์: เลือกฟอร์แมตแฟลช (FAT32 เก็บไฟล์เกิน 4GB ไม่ได้!), คำสั่ง chmod/ls -l ที่เจอทุกวันใน Linux, สร้าง shortcut ไฟล์ = symbolic link — ใช้ในชีวิตประจำวันจริง

J1. File System รับผิดชอบอะไร

จัดการเก็บ/ดึง/จัดระเบียบข้อมูลบน storage — หน้าที่: **ตั้งชื่อ+โครงสร้างไฟล์ · เก็บข้อมูล · ดึง/แก้ไข · Metadata · ความปลอดภัย/สิทธิ์เข้าถึง** · หัวใจคือ **abstraction**: interface เข้าถึงไฟล์เหมือนกันไม่ว่าข้อมูลอยู่บน HDD/SSD/USB (device independence — จาก Quizizz 4 ข้อ 2)

J2. 5 ระบบไฟล์ยอดนิยม

ระบบไฟล์	ไฟล์สูงสุด	Volume สูงสุด	Journaling (สมุดบันทึกเพื่อกู้คืนหลังพัง)	ใช้ที่ไหน
FAT32 (1996)	**4 GB** 😅	8 TB	ไม่มี	USB/การ์ด — เข้ากันได้*ทุก*เครื่อง
NTFS (1993)	16 TB	256 TB	✅	Windows — สิทธิ์/เข้ารหัส (EFS)/compression/quota
ext4 (2008)	16 TB	1 EB	✅	Linux หลัก (ต่อจาก ext3)
exFAT (2006)	16 EB	128 PB	ไม่มี	USB/SD ใหญ่ข้าม OS — แก้ขีดจำกัด 4GB
APFS (2017)	8 EB	8 EB	✅ + Snapshots	Apple ทั้งหมด — เข้ารหัสทั้งดิสก์

🎯 จาก Quizizz 4

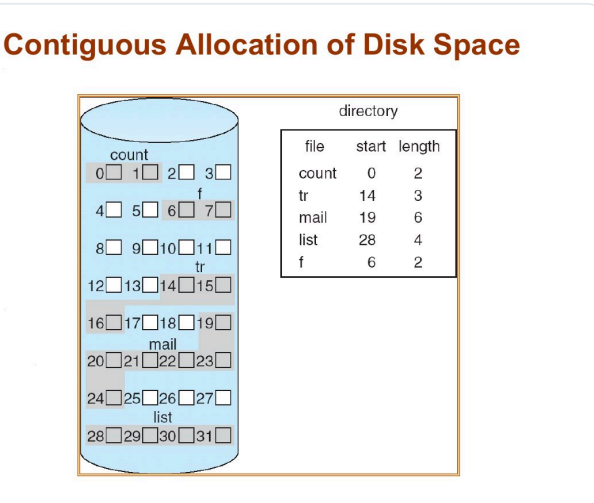
• USB/external drive = **ExFAT** · • **Journaling ≠ เข้ารหัส!** — journaling คือบันทึก log ก่อนเขียนจริงเพื่อ**กู้คืนหลัง crash** · เข้ารหัสของ NTFS คือ **EFS** แยกกัน · • FAT32 ไฟล์เดี่ยว**ต้น 4GB เด็ดขาด** (ฟิลด์ 32 บิต) ดิสก์ว่างเยอะก็ไม่ช่วย

J3. File (ไฟล์) Allocation Methods 🔥 (วิธีจัดสรรบล็อกให้ไฟล์)

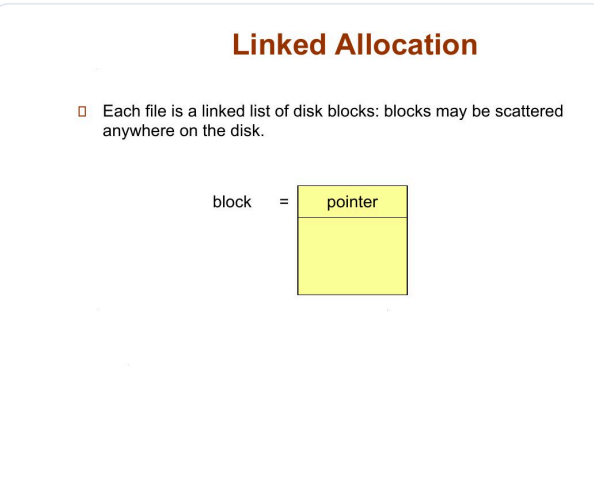
วิธี	หลักการ	ข้อดี	ข้อเสีย
Contiguous	บล็อก**ติดกันเป็นผืน** · directory เก็บ start + length	อ่านเร็วมาก (ไม่ต้อง seek)	**External fragmentation** + ขยายไฟล์ยาก (บล็อกถัดไปถูกจอง = ต้องย้ายทั้งไฟล์)
Linked	บล็อกกระจาย + pointer (ตัวชี้ตำแหน่ง) ชี้บล็อกถัดไปเป็นโซ่ · directory เก็บ start + end	ไม่มี external frag · ขยายง่าย	**Random access ช้ามาก** (ไล่โซ่) · pointer เปลืองพื้นที่ · pointer พัง = ท่อนหลังหาย
Indexed	รวม pointer ทุกตัวไว้ใน **Index Block** · directory ชี้มาจุดเดียว	**Random access เร็ว** · ไม่มี external frag	เปลืองบล็อก index (ไฟล์จิ๋วก็เสีย 1 บล็อก) · ไฟล์ใหญ่ต้อง multi-level

🎯 FAT (File Allocation Table (ตาราง))

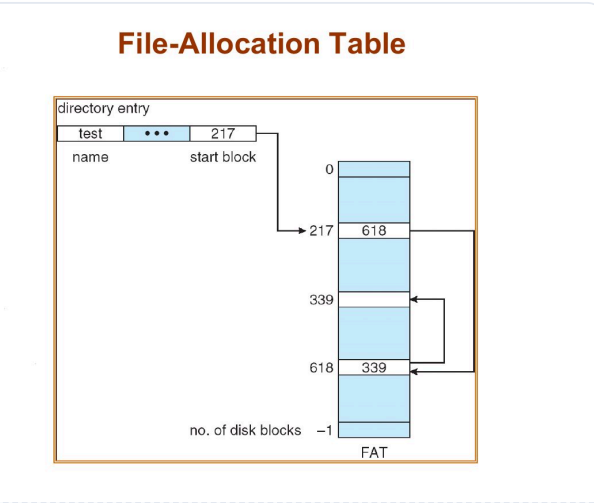
FAT = ดึง pointer ทั้งหมดออกจาก data block มารวมไว้ใน**ตารางเดียว (อยู่ใน RAM)** → ไล่โซ่ได้เร็วโดยไม่ต้องเปิดดิสก์ทีละบล็อก — นี่คือที่มาของชื่อ FAT filesystem



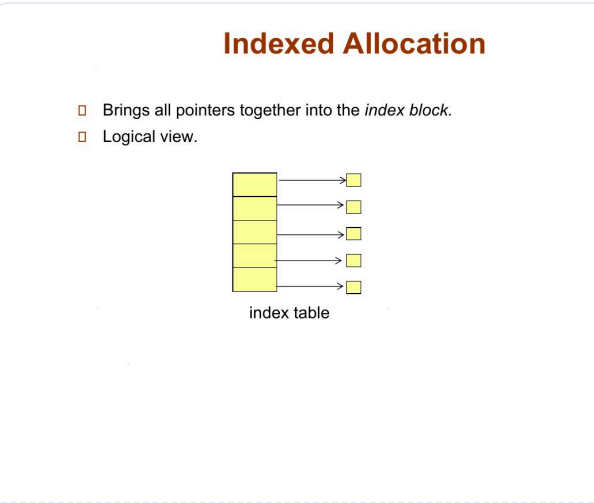
สไลด์ p.19 Contiguous Allocation (การจัดสรรบล็อกติดกัน)



สไลด์ p.20 Linked Allocation (การจัดสรรแบบโซ่ pointer)



สไลด์ p.24 File-Allocation Table



สไลด์ p.25 Indexed Allocation (การจัดสรรแบบสารบัญ)

J3.1 คำนวณ index / inode 🔥 (สูตรเดียวใช้ทุกโจทย์)

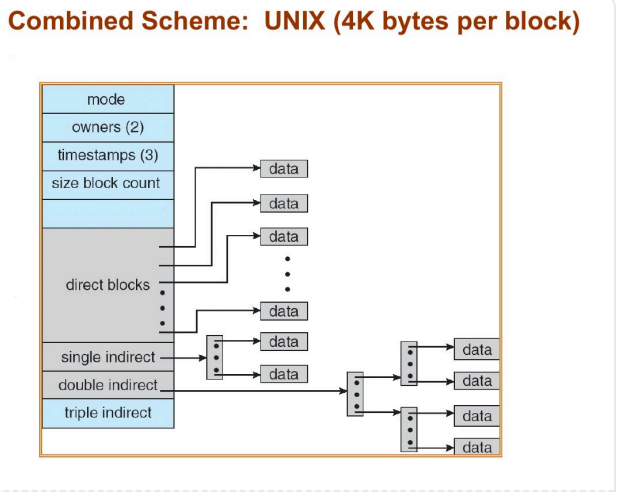
`Entries ต่อ 1 index block = Block Size ÷ Pointer Size`

จาก Quizizz 4: บล็อก 2KB ÷ pointer 4B = **512 entries** → ไฟล์ใหญ่สุด single-level = 512 × 2KB = 1MB

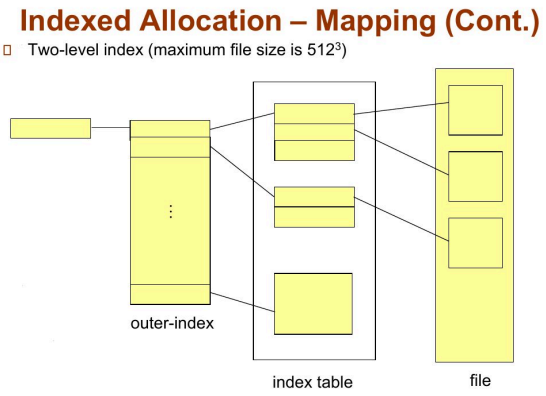
UNIX Combined Scheme — inode (สไลด์ p.29, block 4KB, pointer 4B): มี pointer 15 ตัว

Pointer	จำนวน	คำนวณ	ขนาด
Direct ชี้ data ตรง ๆ	12	12 × 4KB	**48 KB**
Single indirect	1	(4KB÷4B) × 4KB = 1024 × 4KB	**4 MB**
Double indirect	1	1024^2 × 4KB	**4 GB**
Triple indirect	1	1024^3 × 4KB	**4 TB**
ไฟล์ใหญ่สุด ≈ 48KB + 4MB + 4GB + 4TB			**≈ 4 TB**

> **🎯 จุดพลาด**
> ลืมหา "1024 ช่องต่อบล็อก" ก่อน / ลืมบวก direct 48KB / บวกเลขผิดหลัก (4MB ≠ 4×1024KB=4096KB ก็คือ 4MB ตรงกัน แต่ double = 1024×1024×4KB = 4GB)



สไลด์ p.29 Combined Scheme: UNIX (4K/block) — 15 pointers (ตัวชี้ตำแหน่ง)



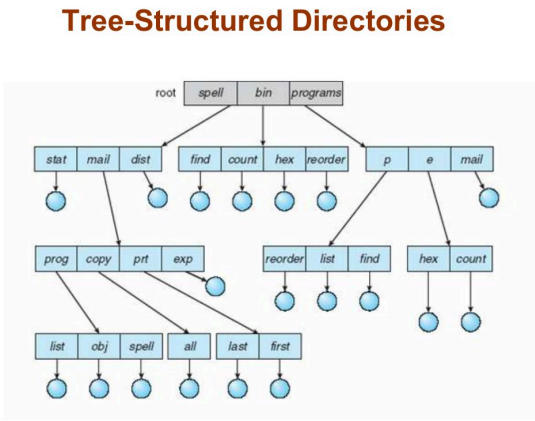
สไลด์ p.28 Two-level index (512^3 ในสไลด์ = block 512B)

J4. Directory (โฟลเดอร์/สารบัญไฟล์) · Tree vs Acyclic Graph

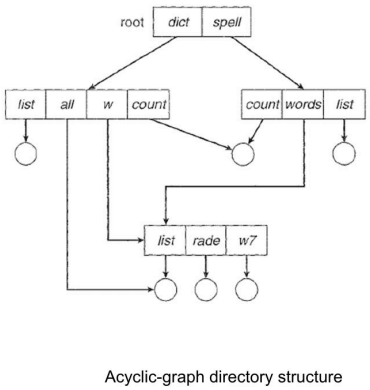
- **Directory** = symbol table แปลง "ชื่อไฟล์ → ข้อมูลไฟล์" · operations: search, create, delete, list, rename, traverse
- **Tree**: ไฟล์มี parent เดียว — ปลอดภัยแต่**แชร์ไม่ได้**
- **Acyclic graph** (ไม่มีวงวน): แชร์ได้ผ่าน **link** — ปัญหา: aliasing (ไฟล์เดียวหลายชื่อ), การลบไฟล์แชร์

การลบไฟล์แชร์ 2 แนวทาง

แนวทาง	ทำอย่างไร	ข้อควรระวัง
ลบเมื่อไรก็ได้	ลบไฟล์ทันทีที่ใครสั่ง	**Dangling pointer** — link ที่เหลือชี้หาอะไรที่ไม่มีแล้ว (แนวทาง symbolic link)
เก็บจน reference หมด	นับ **reference count**: เพิ่ม link → count+1, ลบ link → count−1, **count = 0 ค่อยปล่อยพื้นที่**	ต้องตรวจว่ากราฟไม่มีวงวน (cycle) — แนวทาง hard link ของ UNIX



สไลด์ p.32 Tree-Structured Directories



สไลด์ p.33 Acyclic-graph Directory

J5. Hard Link (ลิงก์ชี้ inode เดียวกัน) vs Symbolic (Soft) Link (ลิงก์ (ทางเชื่อม)) 🔥

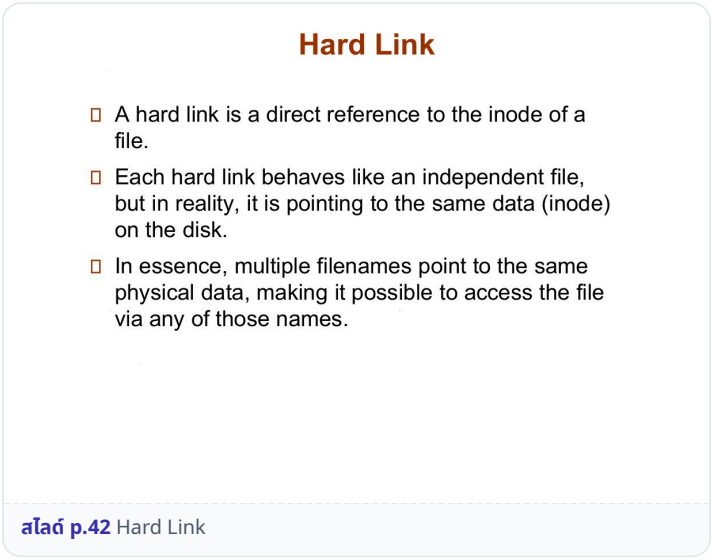
	Hard Link `ln original new`	**Symbolic Link (ลิงก์แบบเก็บ path (shortcut))** `ln -s original new`
ชี้ไปที่	directory entry ชี้**inode เดียวกัน**โดยตรง + เพิ่ม link count	**ไฟล์ใหม่แยกต่างหาก** เก็บ "path" ของต้นทาง (shortcut)
ลบต้นฉบับแล้ว	ข้อมูลยังอยู่ — ปล่อยจริงเมื่อ **link count = 0**	กลายเป็น **broken/dangling link**
ข้าม partition/FS	❌ ห้าม	✅ ได้
ชี้ directory	❌ ห้าม (กัน cycle)	✅ ได้

🏠 อุปมา

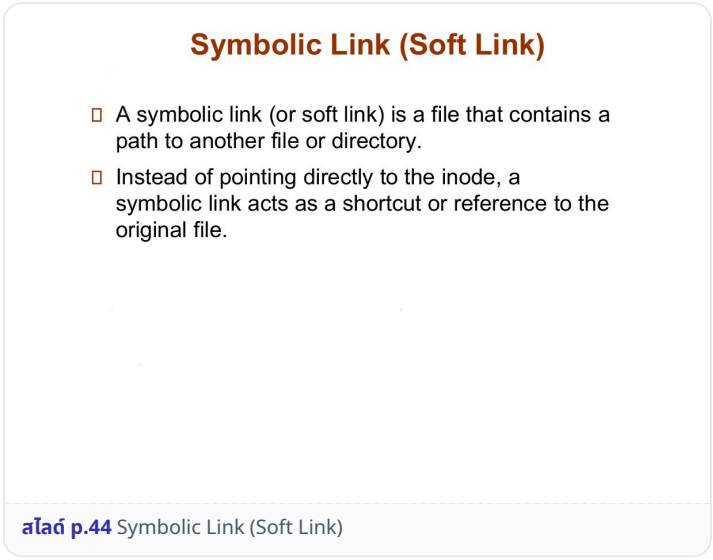
Hard link = ลงทะเบียนบ้านเพิ่มอีกชื่อ — บ้านหลังเดิม อยู่ต่อจนคนทะเบียนหมด · Soft link = กระดาษโน้ตเขียนว่า "บ้านอยู่ซอย 5" — โยนบ้านทิ้ง กระดาษก็ชี้อากาศ

🎯 จาก Quizizz 4

• Symbolic link อีกชื่อคือ **soft link** ไม่ใช่ hard link · • Hard link จัดการด้วย **Reference Counter** ✓ · • **ไม่มีคำสั่ง hardln!** — จริงคือ `ln` (hard) / `ln -s` (symbolic)



สไลด์ p.42 Hard Link



สไลด์ p.44 Symbolic Link (Soft Link)

J6. Mounting (การติดตั้งดิสก์เข้าระบบไฟล์)

- **Mounting** = ติด filesystem ของอุปกรณ์ (partition/USB/network) เข้า directory tree ที่ **mount (ติดตั้งเข้าระบบไฟล์) point** เช่น `mount /dev/sdb1 /mnt/usb`
- ก่อนถอดต้อง `umount` — ให้การเขียนค้าง ๆ เสร็จก่อน ไม่งั้นข้อมูลพัง
- จาก Quizizz 4: อุปกรณ์เดียว **mount ได้หลาย mount point** (ข้อความว่า "ไม่สามารถ" = เท็จ)

File System Mounting

- Mounting a file system refers to making a file system accessible for users and programs on a computer.
- In practical terms, it means attaching a storage device (such as a hard drive, SSD, USB drive, or network file system) to a specific location (or directory) in the system's directory tree so that its contents can be accessed.

สไลด์ p.46 File System Mounting

How Mounting Works?

- Device: A physical device, like a hard disk or a partition on that disk, must be mounted to access its contents.
- The device typically has an identifier like /dev/sda1 for a specific partition of the hard disk.
- Mounting Command: In Linux/Unix systems, the mount command is used. The basic syntax is:
  - mount [OPTIONS] DEVICE MOUNT_POINT
  - Example: *mount /dev/sdb1 /mnt/usb*
    - This command mounts the device /dev/sdb1 (a USB drive in this case) to the directory /mnt/usb.
- Unmounting Command: When you're done with the device, you unmount it with the umount command:
  - *umount /mnt/usb*

สไลด์ p.48 How Mounting Works? (mount/umount command)

J7. Protection — สิทธิ์ RWX 🔥

เจ้าของไฟล์คุมได้ 2 อย่าง: **what can be done** (read/write/execute) กับ **by whom** (owner/group/others)

$$R = 2^2 = 4 \cdot W = 2^1 = 2 \cdot X = 2^0 = 1$$

3 กลุ่ม: owner, group, other (public) — ตัวอย่างสไลด์ p.50: owner=7 (rwx), group=6 (rw-), public=1 (--x) = 761

เลข	บวกได้จาก	แปลง
7	4+2+1	rwx — ทุกสิทธิ์
6	4+2	rw-
5	4+1	r-x (ไม่ใช่ 6! จาก Quizizz)
4	4	r--
0	—	--- ไม่มีสิทธิ์

Access Lists and Groups

- Mode of access: read, write, execute
- Three classes of users

			RWX
a) **owner access**	7	⇒	1 1 1
b) **group access**	6	⇒	RWX 1 1 0
c) **public access**	1	⇒	RWX 0 0 1

สไลด์ p.50 Access Lists and Groups (RWX 7 6 1)